

Frontend Interview Handbook — Part 10: Rapid Revision (100 Most Likely Questions)

Use this the morning of the interview. Read each question, answer out loud from memory in 1-2 sentences, then check yourself against the answer. Anything you hesitate on — go find the full version in Parts 1-9.

JavaScript (1-15)

1. **What's a closure?** A function that retains access to its outer scope's variables even after the outer function returns.
2. `var` vs `let` vs `const`? `var` is function-scoped and hoisted as `undefined`; `let`/`const` are block-scoped with a temporal dead zone; `const` prevents reassignment (not deep immutability).
3. `==` vs `===`? `===` compares value and type with no coercion; `==` coerces types first — always default to `===`.
4. **What's the event loop?** Mechanism that lets single-threaded JS handle async work — microtasks (Promises) drain before the next macrotask (`setTimeout`) runs.
5. **Write debounce.** Clear and reset a timer on every call; only fire after a pause. (Know this cold, from memory.)
6. **Write throttle.** Fire immediately, then block further calls until an interval elapses.
7. `map` vs `filter` vs `reduce`? `map` transforms 1:1; `filter` selects a subset; `reduce` folds into a single accumulated value.
8. **What's `this`?** Determined by how a function is called, not where defined; arrow functions inherit `this` lexically.
9. `null` vs `undefined`? `undefined` = not assigned; `null` = intentionally set to "no value."
10. **What's hoisting?** Declarations are processed before execution; `var` hoists as `undefined`, `let`/`const` hoist but stay in a "temporal dead zone."
11. `Promise.all` vs `Promise.allSettled`? `all` rejects immediately if any promise fails; `allSettled` waits for all and reports individual outcomes.
12. **What's prototypal inheritance?** Objects link to a prototype object; property lookups walk up the chain if not found on the object itself.
13. `call`/`apply`/`bind`? All set `this` explicitly; `call`/`apply` invoke immediately (args individually vs as array), `bind` returns a new bound function.
14. **Shallow vs deep copy?** Shallow copies top-level only, nested objects still shared by reference; deep copy duplicates every level (`structuredClone`).
15. **What's currying?** Transforming a multi-arg function into a chain of single-arg functions.

TypeScript (16-22)

16. `interface` vs `type`? Both describe shapes; `interface` supports declaration merging, `type` handles unions/intersections that `interface` can't.
17. **What are generics?** Reusable, type-safe code that works across multiple types without losing type information (`function useFetch<T>()`).
18. `any` vs `unknown`? `any` disables type checking entirely; `unknown` requires narrowing before use — always prefer `unknown`.
19. **What's `Partial<T>`?** Makes all properties of `T` optional — useful for update/patch functions.
20. **What's `Pick`/`Omit`?** `Pick` selects specific keys into a new type; `Omit` excludes specific keys.
21. **How do you type React children?** `React.ReactNode` — broader than `JSX.Element`, accepts strings/arrays/null too.
22. **When is `any` acceptable?** Rarely — untyped third-party libraries as a last resort, narrowed immediately at the boundary.

React (23-40)

23. **What's the virtual DOM for?** A lightweight in-memory tree diffed against the previous version to minimize expensive real DOM writes.
24. `useEffect` **mental model?** "Synchronize with an external system when these dependencies change" — not literally mount/unmount.
25. **Controlled vs uncontrolled inputs?** Controlled = value driven by React state via `onChange`; uncontrolled = DOM manages its own value, read via `ref`.
26. **Why not use array index as key?** Breaks if list reorders/filters — React can misattribute internal state to the wrong row.
27. `useMemo` vs `useCallback`? `useMemo` memoizes a computed value; `useCallback` memoizes a function reference (for stable identity).
28. **What's prop drilling, and the fix?** Passing props through layers that don't need them; fix with Context or component composition.
29. **What's reconciliation?** React's $O(n)$ diffing heuristic comparing new vs old virtual DOM trees to compute minimal real DOM changes.
30. `React.memo` **purpose?** Skips re-rendering a component if its props are shallow-equal to the last render.
31. **Rules of Hooks?** Only call at the top level, only from components/custom hooks — never in loops/conditions.
32. **What's a custom hook?** A `use`-prefixed function composing other hooks to encapsulate reusable stateful logic.

- 33. **Why does `useEffect` run twice in dev?** React Strict Mode intentionally double-invokes to surface missing cleanup — dev only, not production.
- 34. **State vs props?** Props are passed in and read-only from the child; state is owned/managed internally.
- 35. **What's lifting state up?** Moving shared state to the closest common ancestor so multiple children stay in sync.
- 36. **What's a fragment for?** Groups elements without adding an extra wrapper DOM node.
- 37. **Why avoid inline object/array literals as props to memoized children?** New reference every render defeats `React.memo`'s shallow comparison.
- 38. **What triggers a re-render?** State change, prop change, parent re-render (unless memoized), or context value change for consumers.
- 39. **What's the difference between `key` and `id`?** `key` is React-internal for reconciliation; `id` is a DOM/HTML attribute — don't confuse the two, though they're often the same value.
- 40. **How do you avoid unnecessary re-renders in a large list?** Memoize row components, stabilize callback/prop references, consider virtualization for very large lists.

Next.js & Rendering (41-52)

- 41. **CSR vs SSR vs SSG vs ISR — one line each?** CSR: render in browser after JS loads. SSR: render HTML per request on server. SSG: render HTML once at build time. ISR: static pages regenerated on an interval/on-demand.
- 42. **Why does CSR hurt SEO?** Crawlers/AI retrieval may not fully execute JS or wait for it; content isn't reliably present in initial HTML.
- 43. **What's hydration?** React attaching to server-rendered HTML in the browser, reusing DOM nodes and wiring up interactivity.
- 44. **What's a hydration mismatch?** Server-rendered HTML differs from what client would render (e.g., `Date.now()` in render) — causes React to discard and re-render.
- 45. **What are React Server Components?** Components that render entirely server-side and never ship JS to the client — reduces bundle size.
- 46. **RSC vs traditional SSR — the key difference?** SSR still ships full component JS for hydration; RSC components never ship JS at all unless marked `"use client"`.
- 47. **App Router vs Pages Router?** App Router: RSC by default, nested layouts, fetch-based caching. Pages Router: client-hydrated by default, `getStaticProps`/`getServerSideProps`.
- 48. **How does `next/image` help Core Web Vitals?** Responsive sizing, lazy loading, modern formats, requires explicit dimensions (prevents CLS).
- 49. **What's `generateStaticParams` for?** Pre-renders specific dynamic routes at build time — used for programmatic SEO to pre-build high-value pages.

50. `getStaticProps` vs `getServerSideProps`? Build-time (cacheable, fast) vs per-request (needed for request-specific/personalized data).
51. **What's streaming SSR / Suspense for?** Sends HTML in chunks as ready, so fast content shows immediately while slower parts stream in.
52. **Can Server Components use `useState`?** No — hooks require a Client Component (`"use client"`).

HTML & CSS (53-62)

53. **Why use semantic HTML?** Conveys meaning to screen readers/crawlers, gives free accessible behavior, better than generic `<div>`s.
54. `<section>` vs `<article>` vs `<div>`? `<article>` = self-contained/syndicable content; `<section>` = thematic grouping usually with a heading; `<div>` = no semantic meaning.
55. **Explain the box model.** Content → padding → border → margin; `box-sizing: border-box` makes width/height include padding+border.
56. **Flexbox vs Grid?** Flexbox = one-dimensional (row or column); Grid = two-dimensional (rows and columns together).
57. `em` vs `rem`? `em` relative to parent/self font size (compounds); `rem` relative to root font size (predictable).
58. **Mobile-first vs desktop-first?** Mobile-first (base styles + `min-width` queries) generally preferred — aligns with Google's mobile-first indexing.
59. **What causes CLS, and the fix?** Images without dimensions, font swap, late-injected content — fix with explicit `width`/`height`/`aspect-ratio`.
60. `position: sticky` vs `fixed`? `sticky` acts relative until a scroll threshold, then behaves fixed within its container; `fixed` is always relative to viewport.
61. **What's `async` vs `defer` on scripts?** `async` runs as soon as downloaded (order not guaranteed); `defer` runs after parsing, in document order.
62. **What's `alt` text for, and what makes it good?** Screen reader/SEO description of image purpose in context — concise, not keyword-stuffed.

SEO / AEO / GEO (63-75)

63. **On-page vs technical vs off-page SEO?** On-page: content/HTML factors. Technical: crawlability/infra. Off-page: backlinks/external signals.
64. **What's a sitemap for?** Machine-readable URL list helping crawlers discover pages, especially at scale.
65. `robots.txt` **common mistake?** Blocking JS/CSS needed to render, or confusing "disallow crawl" with "prevent index" (use `noindex` for the latter).
66. **What's canonicalization?** A tag telling search engines which URL version is authoritative among duplicates.

- 67. **Real 404 vs soft 404?** Real 404 returns HTTP 404 status; soft 404 shows "not found" content but returns 200 — worse, since crawlers may index it.
- 68. **What's JSON-LD structured data for?** Explicitly tells search engines/AI what type of entity a page represents, improving rich results/citation eligibility.
- 69. **AEO vs GEO?** AEO = optimizing to be the direct answer (featured snippets, voice, AI Overviews). GEO = optimizing to be cited by generative AI systems synthesizing from multiple sources.
- 70. **Biggest frontend lever for AEO/GEO?** Ensuring content is actually present in server-rendered/static HTML (SSR/SSG), not injected only by client JS.
- 71. **What's crawl budget?** Finite crawling attention a search engine allocates per site — wasting it on thin/duplicate pages hurts high-value page discovery.
- 72. **301 vs 302 redirect?** 301 = permanent, passes ranking signal; 302 = temporary, signal usually stays with original URL.
- 73. **What's thin content?** Low-value, minimally differentiated pages — a real risk in programmatic SEO if not curated carefully.
- 74. **What's E-E-A-T?** Experience, Expertise, Authoritativeness, Trustworthiness — quality framework increasingly relevant to AI source credibility too.
- 75. **Good title tag length?** ~50-60 characters, unique per page, natural keyword placement, no stuffing.

Performance & Core Web Vitals (76-85)

- 76. **Name the 3 Core Web Vitals.** LCP (loading), CLS (visual stability), INP (responsiveness).
- 77. **LCP target and top cause of failure?** Under 2.5s; usually slow server response, render-blocking resources, or unoptimized hero image.
- 78. **CLS target and top cause?** Under 0.1; usually images/embeds without reserved dimensions.
- 79. **INP target and top cause?** Under 200ms; usually long main-thread JS tasks or heavy hydration.
- 80. **What's code splitting?** Breaking one bundle into smaller chunks loaded on demand (`React.lazy`, Next.js automatic route splitting).
- 81. **Preload vs prefetch?** Preload = definitely needed soon, high priority; prefetch = probably needed next, lower priority.
- 82. **Reflow vs repaint?** Reflow recalculates layout/geometry (expensive, can cascade); repaint just redraws pixels (cheaper).
- 83. **Why animate `transform`/`opacity` over `width`/`top`?** Avoids triggering reflow, uses cheap GPU compositing instead.
- 84. **What's a CDN for?** Serves cached content from edge servers near the user, reducing latency.

85. **Tool to measure real-user Core Web Vitals?** `web-vitals` library + RUM, or CrUX data in PageSpeed Insights/Search Console (not just lab-only Lighthouse).

APIs, State, Security (86-96)

87. **PUT vs PATCH?** PUT replaces the whole resource; PATCH applies a partial update.

88. **401 vs 403?** 401 = not authenticated (needs login); 403 = authenticated but not permitted.

89. **What problem does React Query solve?** Caching, dedup, background refetch, loading/error state, race condition handling — vs hand-rolled `useEffect` fetching.

90. **What's a data-fetching race condition?** A slower earlier request resolving after a newer one, showing stale data — fixed via `AbortController` or query-key-based cancellation.

91. **Local state vs Context vs Redux — quick rule?** Local by default; Context for low-frequency shared data; Redux/Zustand for frequent, widely shared, complex state.

92. **What's optimistic UI?** Update UI immediately assuming success, roll back on failure — good for low-stakes, reversible actions.

93. **How does React prevent XSS by default?** JSX escapes rendered values as text automatically; `dangerouslySetInnerHTML` reintroduces the risk if unsanitized.

94. **What's CSRF, and a common defense?** Forged cross-site requests using a user's session cookie; defended with CSRF tokens and `SameSite` cookies.

95. **Where should auth tokens NOT be stored, and why?** Avoid `localStorage` if possible — readable by any JS, including from XSS; prefer `httpOnly` cookies.

96. **CORS — what is it, why does it exist?** Browser mechanism blocking cross-origin requests unless explicitly allowed by the server — prevents malicious cross-site requests.

Testing, Git, CI/CD (97-100+)

97. **RTL's core philosophy?** Test behavior as a user experiences it (queries by role/text), not internal implementation details.

98. `merge` vs `rebase`? Merge preserves full branch history with a merge commit; rebase replays commits for a linear history but rewrites hashes (don't rebase shared branches).

99. `npm ci` vs `npm install`? `npm ci` installs exactly from the lockfile, deletes `node_modules` first, deterministic — used in CI. `npm install` can update the lockfile — for local dev.

100. **What's a feature flag for?** Runtime toggle to ship code without releasing it to users yet, or roll out gradually — decouples deploy from release.

Final Morning-Of Checklist

- ☐ Say the LCP/CLS/INP definitions out loud without notes
 - ☐ Say the CSR/SSR/SSG/ISR explanation out loud without notes — this is the highest-probability deep-dive topic
 - ☐ Write debounce and throttle from a blank file, no copy-paste
 - ☐ Re-read your 6 STAR stories once (Part 9) — especially the failure story
 - ☐ Pick your 2-3 questions to ask the interviewer (Part 9, Section E)
 - ☐ Test your video/audio setup 15 minutes before the call
-

This concludes the 10-part Frontend Interview Handbook. All parts are saved as separate files — combine them into one document if you'd like a single reference, or keep them split by topic for focused review sessions.